

Mini Project Report on

**Hardware Optimization for ECG Signal Processing and Arrhythmia
Classification Using FPGA-Accelerated Machine Learning and
Preprocessing**



*Submitted for mini project of FPGA laboratory as partial fulfilment of B.Tech
Course*

By

1. Gadde Srinivasa Ramanujam-23ECB0A01

2. Budida Karthik- 23ECB0A02

3. Nasika Sasank - 23ECB0A03

Under the guidance of

Prof. P Prithvi Mam

Prof. Vadthiya Narendar Sir

**DEPARTMENT OF ELECTRONICS AND COMMUNICATION
ENGINEERING**

NATIONAL INSTITUTE OF TECHNOLOGY, WARANGAL

Telangana-506004

March-2025

Certificate

This is to certify that the Project work entitled “**Hardware Optimization for ECG Signal Processing and Arrhythmia Classification Using FPGA-Accelerated Machine Learning and Preprocessing**” is a Bonafide record of work carried out by Gadde Srinivasa Ramanujam – 23ECB0A01, Budida Karthik – 23ECB0A02, Nasika Sasank – 23ECB0A03 submitted to the faculty of Department of Electronics and Communication Engineering, in partial fulfilment of the requirements for the award of the degree of Bachelor of Technology in Electronics and Communication Engineering for the course EC257- FPGA Based design Lab at National Institute of Technology, Warangal during the academic year 2024-2025.

Dr. Vakula Head of the department Electronics and communication dept. National Institute of Technology Warangal	Dr. Prithvi Associate Professor Electronics and communication dept. National Institute of Technology Warangal	Dr. Vadthiya Narendar Assistant Professor Electronics and communication dept National Institute of Technology Warangal

Index

1. Abstract
2. Content
 - 2.1. List of figures
 - 2.2. List of abbreviations
 - 2.3. List of tables
3. Introduction
4. Literature Survey
5. Content
 - 5.1. Problem statement
 - 5.2. Motivation
 - 5.3. Novelty
 - 5.4. Implementation flow
 - 5.5. Objectives
6. Theory
7. Discussion and Conclusion
 - 7.1. Results and conclusion
 - 7.2. Future work
8. Appendix: Code
9. References

Abstract

Arrhythmia is a common medical condition that is essentially irregularity in the heartbeat. This involves a heart rate that can be too slow, too fast, or any other type of irregularity.

The symptoms of Arrhythmia range from dizziness, shortness of breath, shortness of breath and so on.

The increased amount of stress and sedentary lifestyle in the modern era combined with the increase in air pollution increased the likelihood of arrhythmia. Additionally, modern diets which contain high amounts of processed food contribute to the risk.

Arrhythmia in general can be quite harmless. However, untreated it may be fatal and cause heart failures, heart strokes and cardiac arrests.

Diagnosis of Arrhythmia involves utilizing ECG processing, Holter Monitor, Implants and so on. However, these methods are very slow and costly and also power intensive. This leads to complications in the diagnosis process.

In this project, we propose a new methodology to classify Arrhythmia in a fast, low power, efficient and accurate manner via a Field Programmable Gate Array (FPGA) board and various machine learning and preprocessing techniques.

Creation of problem specific circuits helps accelerating the evaluation time and power requirements over processing ECG data in the traditional manner.

The implemented solution is evaluated on the MIT-BIH atrial fibrillation data and compared with previous solutions for this issue.

Table of Contents

1.List of Tables

Table 1: Performance Comparison of Different Models on Various Platforms

Table 2: Resource Utilization of MLP Models on FPGA

Table 3: LUT Utilization Analysis: Serial vs. Parallel Preprocessing

Table 4: Accuracy Comparison of pruned and unpruned tree

Table 5: Performance Comparison of Proposed and Existing Models

Table 6: Hardware Utilization Comparison of Different Models and Preprocessing Techniques

Table 7: Performance Analysis: Evaluation Time of MLP vs. Pruned Decision Tree

2.List of figures:

Figure 1: Implementation flowchart

Figure 2: Serial in parallel out shift register

Figure 3: ASM chart for serial preprocessing

Figure 4: ECG Signal Structure

Figure 5: Simulation Output

Figure 6: Output class displayed on FPGA

Figure 7: Display of input signal

Figure 8: Output class displayed on FPGA

3.List of abbreviations:

- FPGA – Field Programmable Gate Array
- ECG - Electrocardiogram
- MIT-BIH -Massachusetts Institute of Technology and Boston's Beth Israel Hospital
- AFEn – Entropy for Atrial Filibiration
- SampEn – Sample Entropy
- CosEn – Coefficient of Sample entropy
- ADC – Analog to Digital converter
- CCP – Cost complexity path
- FIFO – First in first out
- ASM – Algorithmic state machine

Chapter 1: Introduction

Arrhythmia is a common heart disease that is characterized by improper beating of the heart. Arrhythmia can become critical if not properly treated early.

In India alone, more than 10 million cases of Arrhythmia are diagnosed each year. However, this diagnosis can be very expensive and also increase the excessive workload on medical staff. Considering the shortage of doctors across the world, this is not ideal.

Automating Arrhythmia detection utilizing the advancements in Machine learning, hardware acceleration and lower power consumption technologies can lead to improvements in Arrhythmia treatments.

This work proposes a new approach to deal with arrhythmia detection in an efficient manner.

ECG signals are utilized to find Arrhythmia in heartbeat. (i.e. ECG signals are pre-processed and then classified into various types of Arrhythmia)

Utilizing FPGA to create application specific circuits helps with keeping the processing costs and power consumption to minimal.

Additionally, due to the parallel processing abilities of FPGA, the processing times can be expected to much faster compared to processing on traditional CPUs and GPUs.

Chapter 2: Literature Survey

Work [2] created a new entropy algorithm for finding arrhythmia known as “AFEn” (Entropy for AF). This entropy feature showed a high correlation with the odds of having Arrhythmia.

Work [1] Utilized AutoMLP framework along with various extracted features (meanRR period, maximumRR period, minimumRR period, median heart rate) and entropy features (Sample Entropy, Coefficient of Sample Entropy, Entropy for AF) utilizing a total of 7 features to classify heart beats into two classes (Normal and abnormal).

However, the processing of these entropy features (especially Entropy for AF) is very expensive and is very slow and is not entirely implementable real-time.

Work [3] utilized a mix of data compression algorithms, data preprocessing algorithms, and convolutional neural networks to obtain the output.

While this work gave very good accuracy results, it is a very heavy model overall and will require lots of hardware to implement.

The details of each of the above-mentioned works is shown in the table 1 .

Table 1 Performance Comparison of Different Models on Various Platforms

Reference	Platform	Model	Classification Type	Accuracy
[1]	Xilinx Pynq-Z1	MLP	Binary	94.20%
[2]	Intel I7-8550U	AFEntropy	Binary	94.80%
[3]	Intel I9	CS-CNN	Binary	96.25%

Additionally, paper [1], [4], [5] studied the utilizations of various models and parallel optimization architectures.

The utilizations of the different models and architecture combinations can be seen in below table.

Table 2 Resource Utilization of MLP Models on FPGA

Reference	Model	Architecture	BRAM	DSP	FF	LUT
[1]	MLP 3 Layer	L1U, L2U	0	21	6271	15136
[1]	MLP 5 Layer	L1U, L2U	0	22	7473	24714
[4]	MLP Saric	L1U, L2U	0	21	6698	18556
[5]	MLP Borghi	L1U, L2U	0	21	19454	93682

Although not explicitly mentioned, we can expect the other models such as the CNN network to utilize much more hardware if it were to be implemented on an FPGA or a similar device.

Keep in mind that the above utilization tables are excluding the hardware required to preprocess the ECG signal data in the first place.

Chapter 3: Overview

3.1 Problem Statement

The heart of the project lies in designing and developing an efficient FPGA based system to process ECG signals and classify them into different types of Arrhythmia conditions: -

- **'N' (Normal beat)** → **Class 0** – Regular heartbeats with no abnormalities.
- **'S' (Supraventricular ectopic beat)** → **Class 1** – Premature beats originating above the ventricles.
- **'V' (Ventricular ectopic beat)** → **Class 2** – Premature beats originating in the ventricles, often associated with arrhythmia.
- **'F' (Fusion beat)** → **Class 3** – A combination of normal and abnormal beats occurring together.
- **'Q' (Unknown beat)** → **Class 4** – Beats that do not fit into the other categories, possibly artifacts or unclassified rhythms.

This problem involves various FPGA accelerated steps to classify the signal into one of the five required beat classes.

The final implementation should be light weight both in processing speed and hardware utilization as well as power consumption.

Additionally, the implementation should be robust.

3.2 Motivation

Training doctors is a costly process, with estimates suggesting it can take upwards of **250,000\$ to 500,000\$** (in USA) to fully train a single doctor over several years. At the same time, India faces a shortage of over **90,000** doctors, according to recent healthcare statistics. The lack of medical facilities in rural and underserved areas further compounds this issue, leading to increased patient loads and delayed diagnoses for conditions like Arrhythmia.

This system aims to address these challenges by providing an automated solution for detecting ‘Arrhythmia’ from ECG signals, potentially reducing diagnosis time by a substantial margin and making quality healthcare more accessible. Implementing this project using FPGA-based solutions, in particular, can operate several times faster and power efficient than traditional CPU-based solutions for signal processing and machine learning tasks.

This efficiency leads to significant cost savings, as these systems may operate at a fraction of the expense of traditional diagnostic equipment

By utilizing FPGA technology for parallel processing, this project aims to give new insights in ECG signal processing in an optimized manner.

Additionally, this system may also be utilized in various future embedded medical devices for its light-weight and power efficient implementation.

3.3 Novelty

Most of the approaches that were implemented in previous works utilize using various types of entropy and statistical features to classify heartbeats. Although this approach may seem fine (Even if you exclude the high computation cost and time required), further analysis can clearly prove that these approaches lose a lot of data during the preprocessing causing the output to be misclassified. (i.e. lots of critical information of the signal is being lost during the preprocessing).

Additionally, previous works focus on classifying a large time sample into normal/abnormal beats whereas the current work proposes a new idea to classify each individual heartbeat to the normal/abnormal category hence by decreasing the chance of missing abnormal heartbeats.

Additionally, the decision tree was optimized for FPGA deployment via pruning making it more hardware efficient and less power consuming.

Due to the application specific nature of the circuit, the preprocessing is also much faster and consumes lower power than traditional methods.

Additionally, the faster computation speeds allow for a better real-time implementation compared to the bulky computations required in previous works.

The proposed system is also scalable to a larger size implementation easily.

3.4 Implementation flow

3.4.1 System Overview

The system was finalized after trying various approaches regarding the problem statement. The initial approach is similar to the one in [1]. This approach utilized various types of **statistical features** (mean RR interval, min RR, max RR, median heart rate, logarithm of mean RR), **entropy-based features** (sample entropy, approximate entropy, permutation entropy, spectral entropy, singular value decomposition (SVD) entropy), and **complexity & fractal-based features** (Higuchi fractal dimension, Hjorth mobility and complexity, Lempel-Ziv complexity).

The features were extracted considering anywhere from 10-30 heartbeats sample window. (Around 10-30 seconds on average)

A multi-layer perceptron was used with a topology of inputSize-12-16-12-2 was used to classify the features into two classes (Normal and abnormal).

This model is not only very bulky due to the large topology and high number of floating-point multiplications but also only gave an accuracy of 91.81%

We can infer from the test results that clearly there is a large of data loss during the preprocessing/feature extraction stage which is causing this variation. (i.e. critical data is being lost).

Hence the 2nd approach involved preprocessing the data in a way that has lesser amount of loss. The ECG signals are considered at a lower sampling frequency of 125Hz. Afterwards, each ECG signal is segmented into involve exactly one beat window. This window is padded to a fixed size and passed through a multi-layer perceptron.

While this acquired a much higher accuracy of 91.97% on multiclass classification, the design was extremely heavy with all the floating point multiplications and cannot be implemented parallelly. Additionally, the parallel approach was found to use an extreme amount CARRY4 cells confirming our suspicions.

With some acceleration algorithms and clocking, the model can be reduced in hardware but this beats the original purpose of decreasing both hardware costs and evaluation time.

Although the model itself was a failure, we can confidently say that the preprocessing was a success due to the increase in the classification accuracy.

The final and proposed approach of this study involves utilizing a pruned decision tree with 12-bit input. The pruned tree helps in preventing overfitting of the data and also decreases the hardware utilization. Additionally, the model is hardware accelerated due to the faster evaluation of decision trees on hardware.

The same preprocessing method was used for this implementation as before.

The data flow pipeline of this approach can be seen below.

3.4.2 Data flow pipeline

The system operates on the below pipeline to classify ECG signals into various types of Arrhythmia.

1. Data input

ECG signals are taken using a general ECG sensor which consists of a simple amplifier and pass filter.

The analog ECG signal is converted into a digital signal using a 12-bit ADC.

This 12 Bit signal is passed to the FPGA board via its PMOD ports.

2. Clock Converter

We need to sample the ECG signal at a fixed pre-defined frequency. In this case, we expect a 125Hz signal (Based on how we trained the model and Nyquist sampling rate).

By default, the internal clock of FPGAs is very fast. We utilize a clock converter to downscale the clock from very high speed to 125Hz.

3. Data Queue

At each clock edge of the now generated slower clock, we sample the PMOD ports to get an ECG signal sample. We save this data sample into a data queue made using shift registers. In this way we get the required live data.

4. ECG Preprocessing

The now acquired live data is preprocessed using various transformation techniques. Finally, the data is preprocessed in the pre-defined manner and sent to the classifier.

5. Decision Tree based classification

A hardware & time optimized decision tree is utilized to classify the preprocessed signal into the possible heart condition.

6. Output display

After acquiring the classified output, the seven-segment module is utilized to display the classified output on the seven-segment display for easier visualization.

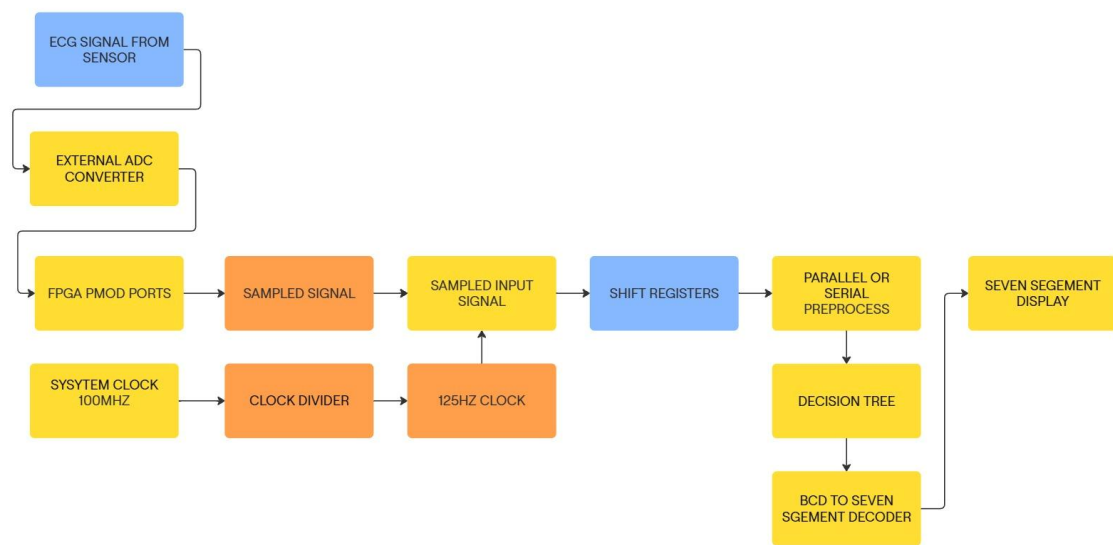


Figure 1 implementation flowchart

3.5 Objectives

- **Context**

Arrhythmia poses significant threat and needs to be diagnosed fast and effectively with high accuracy.

Processing of ECG signals using hardware accelerated designs created using FPGAs can help in this task.

These designs are not only fast but also power efficient and can be used as medical implants, wearable electronics etc.

- **Challenges**

High computation costs of the pre-existing algorithms/designs.

Accurate real-time classification of ECG signal segments.

Low power consumption and high-speed processing.

Light weight and robust hardware design (Minimal cost).

- **Expected outcome**

A light weight and robust hardware accelerated system capable of classifying ECG segments into one of five classes.

Design having low power consumption, very high speed and low hardware requirements.

4 Chapter 4: Theory

4.1 ADC Conversion and Sampling

By default, ECG signals are analog in nature. However, FPGAs deal with digital data and hence cannot process ECG signals as is.

Additionally, the signal should be sampled at a specific frequency. To determine the ideal value of this frequency, we refer to the Nyquist sampling rate of ECG signals.

For generalized medical applications, ECG signals are expected to be sampled at 150-300Hz. For a narrow application such as Arrhythmia detection, a sampling frequency of 125Hz helps in reducing in unnecessary noise in the signal frequency.

To convert the analog values of the ECG sensor output to digital values, we use a simple external ADC.

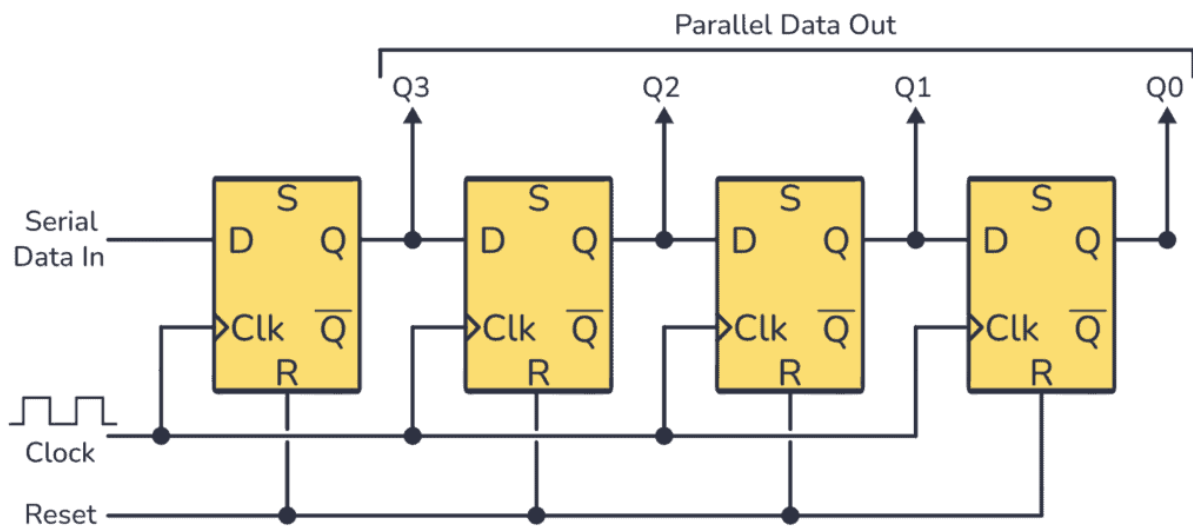
Some FPGA boards contain internal ADC modules which can be utilized for the same purpose. However, this was avoided to maintain a generalized approach independent of board.

In our case, an ESP32 was used as the ADC. The ESP32 contains an onboard 12-bit ADC. This 12 Bit data is passed to the FPGA board via its PMOD ports parallelly. (i.e. 12-bits at same time).

The clock divider was made by simply dividing the internal 100MHz of the Nexys DDR4 board with a factor of 80,000 by using a mod 80,000 counter.

4.2 Data Queueing

The latest ECG data samples obtained from the external ADC are required to be held on the FPGA in a FIFO manner for further processing. For this requirement, we create a n-bit shift register using D-Flip flops. (Note: - Here “N” is the length of the queue and not the ADC resolution).



[6] Figure 2 Serial in parallel out shift register

Reference: <https://www.build-electronic-circuits.com/shift-register/>

12 Of these shift-register based queues are created. Each of the queues is used to create a queue of one of the samples.

Later the queues are pivoted to convert from row-column (12 Samples of N bits) to column-row (N samples of 12-bit) to reobtain the data samples in the required manner.

Here we utilized a serial in parallel out register for the implementation.

This can be replaced with a serial in serial out queue if required to further decrease power consumption if necessary.

4.3 Preprocessing of signals

Preprocessing of Signals Can be done either using serial or parallel

The Goal is to process the ecg signal of length 300 and extract a 187-sample segment from it. This is done by detecting peaks and selecting the segment around first two peaks.

Parallel Implementation:

1.Detection of peaks in ECG Signal:

we will scan through the 300 sample identify local peaks

- A local peak is considered a potential peak if height is greater than or equal to threshold height.

2.Filtering Peaks Based on Distance:

- Some peaks might very close to each other, so we a filter the peaks in such a way that, we get maximum number peaks with distance between any two peaks greater than a fixed distance. Let's call the fixed distance as D
- We follow a simple greedy approach as follows
- Let $P_1, P_2.. P_n$ be the potential candidate peaks detected in first step.
- We process each sample one by one.
- For each peak P_i
 - If distance between P_i and P_{i-1} if less than D then we replace P_i with maximum amplitude of P_i and P_{i-1}
 - Else we just add P_i to peaks

3.Extracting the Signal:

- If number of valid peaks are less than two then then output all zeroes
- now we select the signal segment from first peak to second peak also extra 40 samples
- if the signal is less than 187 then it is padded with zeros

4.Scaling the signal:

- $x_{min} = \min_{i \in S} x_i$
- $x_{max} = \max_{i \in S} x_i$

- $R = \max(1, x_{\max} - x_{\min})$
- Every x_i is scaled in range $[0, 4095]$
- $y_i = (x_i - x_{mi}) \times 4095 \div range$

Serial Implementation:

- Here instead of Parallel processing we use serial and decrease the LUT Utilization
- This is beneficial for FPGA designs with limited resources compared parallel version
- Unlike combinational approach that completes processing in a single cycle this uses the slow clock which was converted from fast clock
- The above ASM chart, represents the serial execution of ECG processing
- Peak Detection, signal processing and scaling happen sequentially.

Table 3 LUT Utilization Analysis: Serial vs. Parallel Preprocessing

Implementation Type	Processing time	LUT Utilization
Parallel	<100ns (estimate)	127945
Serial ASM	Around 675 clock cycles (worst case)	24052

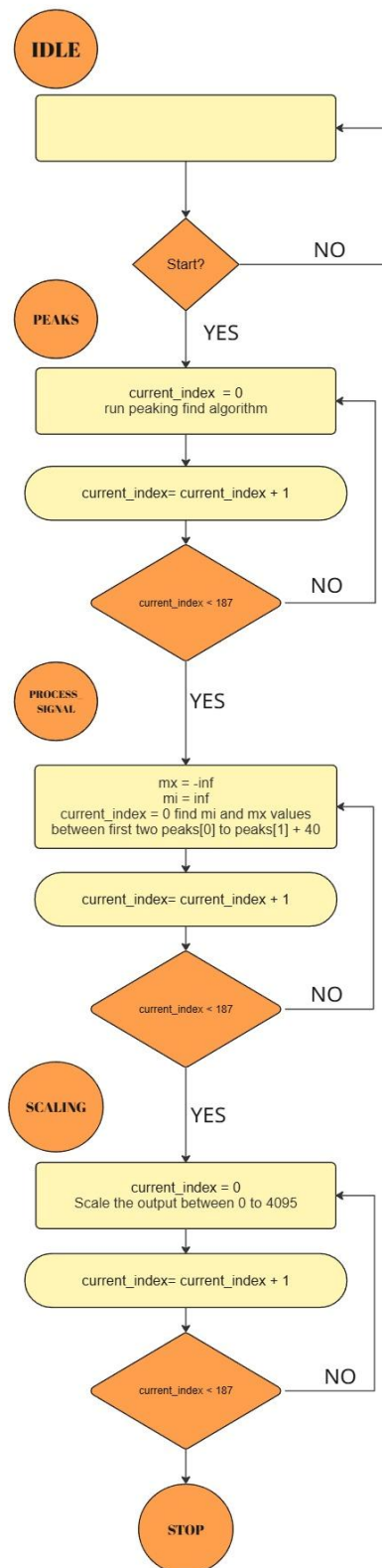


Figure 3 ASM chart for serial preprocessing

4.4 Decision Tree based classification

A decision tree is a machine learning model that classifies input data based on various “decision nodes”. It is called a “Tree” due to its shape. Decision Trees can be used for both classification and regression tasks in general. In this particular study, we utilize them for classification. Traditionally, Decision trees were created by hand. In the current era, we use machine learning to create optimized decision trees using supervised learning.

The MIT-BIH database is used to train the decision tree model and later validate its accuracy.

However, this approach of training a decision tree may yield a lower accuracy than expected and also utilizes a much higher amount of hardware due to the higher amount of decision nodes.

To decrease the hardware used and increase the accuracy, we “Prune” the decision tree using Cost Complexity Pruning.

We choose an ideal CCP-Alpha value by testing the decision tree on a part of the Validation data and check the influence of various nodes on the output. Afterwards, we prune the tree by removing the nodes with a low value of CCP-Alpha (i.e. nodes whose existence is not significant to the decision making).

This not only decreases the amount of hardware used but also increases accuracy by preventing overfitting.

Below table shows the size and accuracy of the tree obtained in our implementation with and without pruning.

Table 4 Accuracy Comparison of pruned and unpruned tree

Tree type	Number of nodes	Accuracy
Unpruned Tree	4711	95.39%
Pruned Tree	1911	96.12%

4.5 ECG Signal structure

ECG signal gives the rhythmic contraction and expansions of the heart as electrical signals.

These signals are generally obtained by sampling the body at various specific points and then passing them through an amplifier followed by a band pass filter.

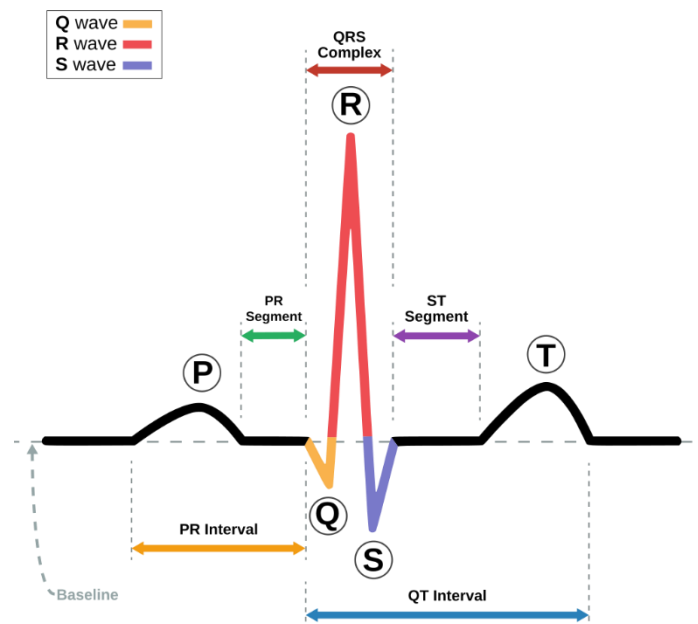


Figure 4 ECG Signal Structure [7]

The major section of an ECG signal is the QRS complex. Generally, beats segmented by taking two consecutive R-peaks.

We followed a similar procedure in this implementation as explained in the preprocessing section.

Chapter 5: Results and Conclusions

5.1 Results & Conclusion

Below table shows the accuracy and classification type of various studies.

Table 5 Performance Comparison of Proposed and Existing Models

Reference	Platform	Model	Classification Type	Accuracy
[1]	Xilinx Pynq-Z1	MLP	Binary	94.20%
[2]	Intel I7-8550U	AFEntropy	Binary	94.80%
[3]	Intel I9	CS-CNN	Binary	96.25%
Proposed	Artix-7 Nexys DDR4	Pruned decision Tree	Multi-Class (Five classes)	96.12%

Though at first sight the proposed work may not seem much different in terms of accuracy, the classification type of the proposed work is multi-class of five classes whereas the previous works focused on binary classification.

Additionally, the amount of preprocessing involved in the previous works is extremely high. This leads to requiring an extreme amount of hardware along with slow processing times.

The below table shows the hardware utilization of some previous works.

Table 6 Hardware Utilization Comparison of Different Models and Preprocessing Techniques

Reference	Model	Architecture	BRAM	DSP	FF	LUT
[1]	MLP 3 Layer	L1U, L2U	0	21	6271	15136
[1]	MLP 5 Layer	L1U, L2U	0	22	7473	24714
[4]	MLP Saric	L1U, L2U	0	21	6698	18556
[5]	MLP Borghi	L1U, L2U	0	21	19454	93682
Proposed	Pruned Decision Tree (No preprocessing)	Not applicable	0	0	0	684
Proposed	Pruned Decision Tree (Serial Preprocessing)	Not applicable	0	0	12978	24052
Proposed	Prune Decision Tree (Parallel preprocessing)	Not applicable	0	0	3900	127945

Here we can see that even after preprocessing the input signal, the hardware utilization of the entire design is much lower than the previous works (Which assumed the signal to be preprocessed externally). This is a massive gain in the results considering the computation costs of the preprocessing steps that were required for these models especially to calculate the entropy-based features.

However, the parallel preprocessing can be seen to have high utilization and cannot be expected to run on most common commercial FPGA devices.

The serial preprocessing utilizes a valid number of resources and can run on most of the commonly available FPGA boards.

The high resource utilization in the previous works can be accredited to the high resource consumption of floating-point multiplications that are required for multi-layer perceptrons,

Table below shows the evaluation time for various designs in previous works.

Table 7 Performance Analysis: Evaluation Time of MLP vs. Pruned Decision Tree

Reference	Classification model	Other factors	Evaluation time
[1]	MLP 3 Layers	No preprocessing	200ns
[1]	MLP 5 Layers	No preprocessing	260ns
Proposed	Pruned Decision Tree	No preprocessing	10.675ns
Proposed	Pruned Decision Tree	Parallel preprocessing	<100ns (estimate)
Proposed	Pruned Decision Tree	Serial preprocessing	Around 675 clock cycles (worst case)

Here we can see that our proposed work shows a massive improvement in the calculation time. (Note that the preprocessing used in [1] is extremely computationally expensive)

On the Nexys 4DDR, the serial preprocessing pruned decision tree can be expected to take around 6.75 micro seconds.

This time can be reduced by applying various approaches to decrease the preprocessing time and can be expected to be optimized to around 1-2 micro seconds (Worst case).

It is very difficult to find the exact time it takes for the output in the parallel preprocessing case but we can estimate it to less than 100ns based on the design. This can still be considered extremely fast and real time considering that the sampling rate of the input signal is 125Hz (Around 1 ms).

Simulation results: -

Result 1 (Parallel preprocessing output) :-

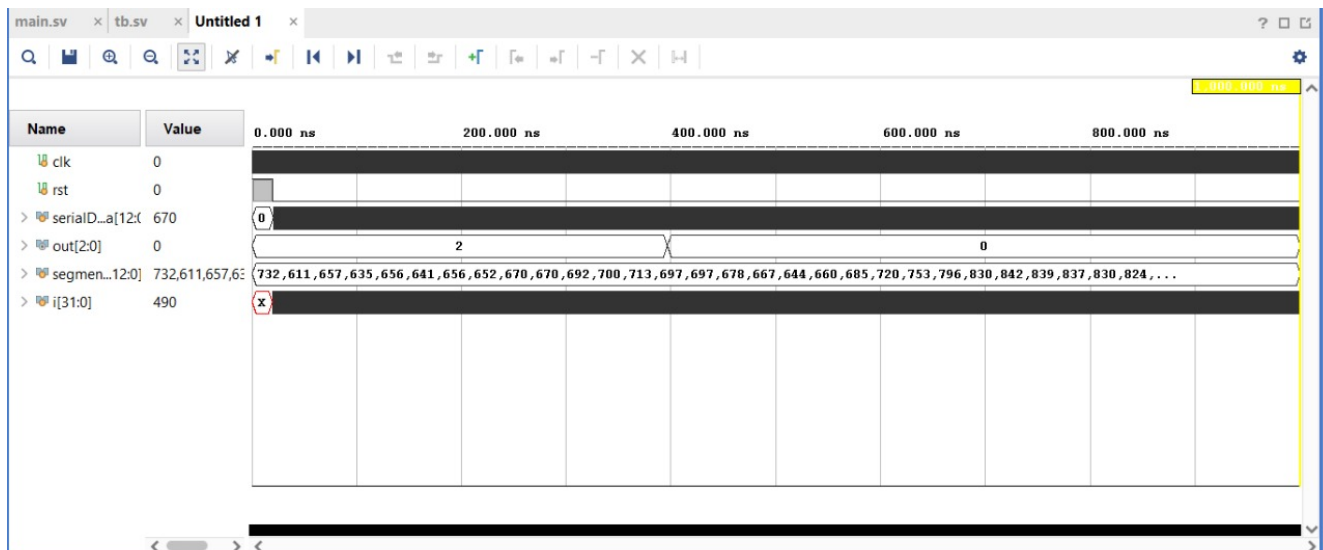


Figure 5 Simulation Output

Data is passed clock by clock to the queue using the serialData 13 bit port (First bit is always 0 just to denote sign). While the input is yet to be filled, the output is unstable (Output comes out as 2 which refers to Ventricular ectopic beat). This is just a consequence of the preprocess output being all 0s and can be ignored.

After the input is filled with enough number of samples, the output becomes the actual prediction (Which is 0 in this case which refers to normal heartbeat).

Result 2 (Example of preprocessed output shown in TCL console): -

```
Min Value: 210, Max Value: 4072, Range: 3862
Scaled processed_signal[0] = 3797
Scaled processed_signal[1] = 1501
Scaled processed_signal[2] = 68
Scaled processed_signal[3] = 276
Scaled processed_signal[4] = 273
Scaled processed_signal[5] = 356
Scaled processed_signal[6] = 396
Scaled processed_signal[7] = 461
Scaled processed_signal[8] = 488
Scaled processed_signal[9] = 507
Scaled processed_signal[10] = 497
Scaled processed_signal[11] = 491
Scaled processed_signal[12] = 466
```

Above image shows the output of the preprocessing module after finishing a preprocessing iteration on a queue. (Same output will be obtained for both parallel and serial preprocessing).

We can see that the preprocessing module considers the minimum value, maximum value and the range to properly normalize the considered segment from 0 to 4095 (Both inclusive).

On-board results: -

Result 1 (Signals are saved onto FPGA and are directly passed) :-

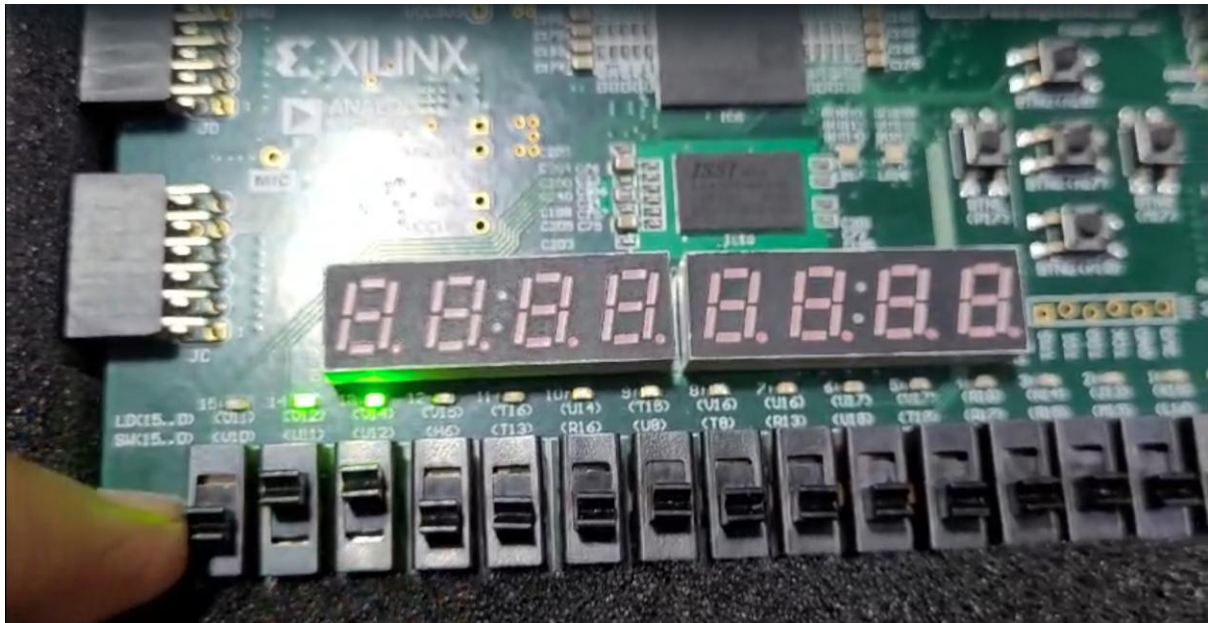


Figure 6 Output class displayed on FPGA

Segments of various ECG sections was saved onto the FPGA before hand as registers and the signals are multiplexed between different types of heart beat segments using switches.

The output class is displayed on the LEDs as shown.

The output matched the expectations.

Result 2 (ESP32 is used to pass heartbeat signals): -

ESP32 is used to pass segments into the FPGA via its PMOD ports. A simple webserver was setup using the ESP32 to output different outputs.

Select Segment to Output

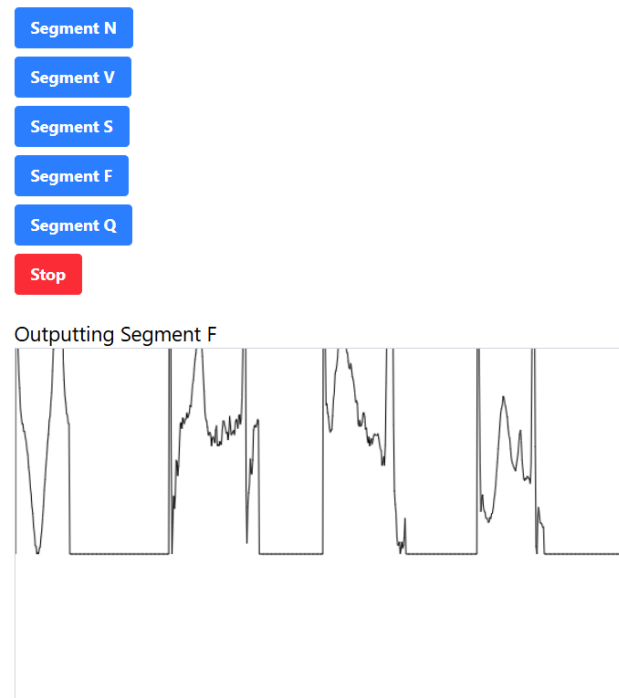


Figure 7 display of input signal from ecg sensor

As both the FPGA and ESP32 are working at 3.3V, we can freely pass data between the devices using a common ground as reference.

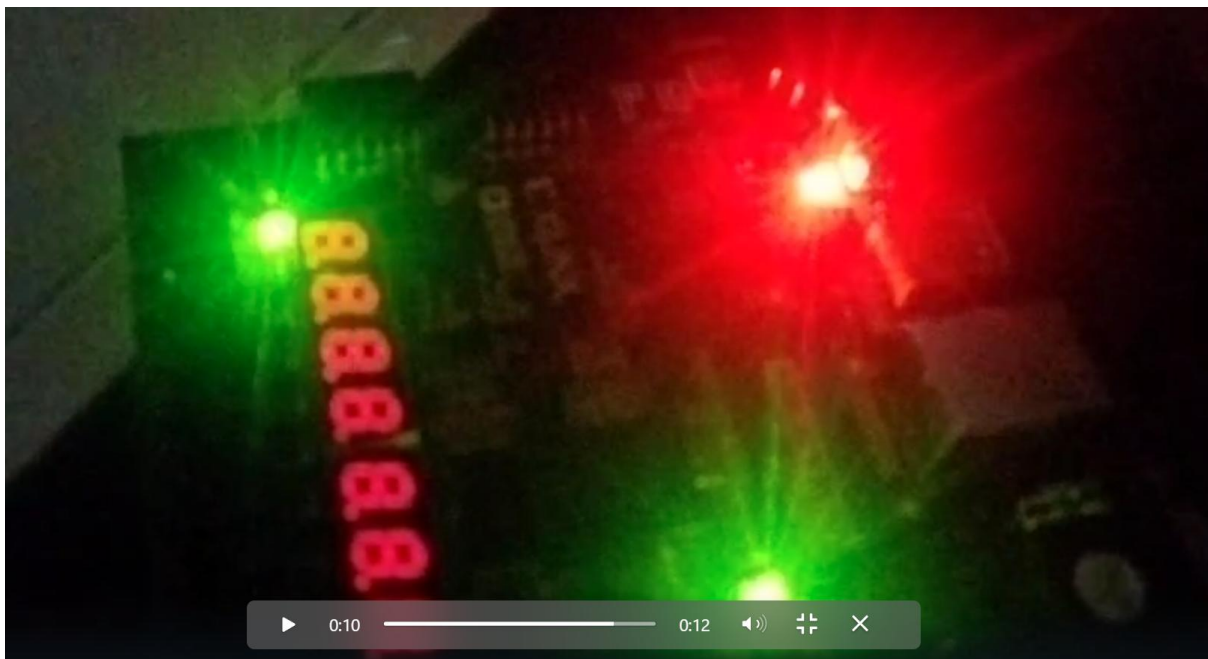


Figure 8 Output class displayed on FPGA as class 2

Above photo shows the output of the FPGA with data being passed from the ESP32.

Due to timing issues, the output was not stable and we were not able to capture it properly.

Further improvements can be made using better timing-based algorithms to get better output. (Such as holding data, using start,busy signals etc etc).

5.2 Future Work

Future developments to this project may involve: -

- Improvising the classification algorithm for higher accuracy
- Decreasing hardware utilization and processing time by improvising the preprocessing algorithms.
- Improvise the preprocessing algorithm making it more robust compared to the current setup.
- Make the design consume even lower power.
- Make the design more robust in real time application.
- Integrate the design with various other mechanisms such as data recording, more control, simpler two-way data transfer protocols etc.
- Implement a simple display to visualize the data better.
- Add additional Digital signal filters to prevent mistakes in peak finding.

Chapter 6: Appendix

Source Codes: -

6.1 Decision Tree Module (Same for all implementations):-

```
package decisionTree_fixpt_pkg;
    typedef logic [12:0] vector_of_unsigned_logic_13;
    endpackage: decisionTree_fixpt_pkg
import decisionTree_fixpt_pkg::*;
module decisionTree_fixpt
    ( input vector_of_unsigned_logic_13 features[0:186] ,
      output logic [2:0] output_rsvd);

    logic [2:0] output_rsvd_1; /* ufix3 */
    logic [2:0] decisionTree_fixpt_tmp; /* ufix3 */
    logic [2:0] decisionTree_fixpt_tmp_0; /* ufix3 */
    logic [2:0] decisionTree_fixpt_tmp_1; /* ufix3 */
    logic [2:0] decisionTree_fixpt_tmp_2; /* ufix3 */
    logic [2:0] decisionTree_fixpt_tmp_3; /* ufix3 */
    logic [2:0] decisionTree_fixpt_tmp_4; /* ufix3 */
    logic [2:0] decisionTree_fixpt_tmp_5; /* ufix3 */
    logic [2:0] decisionTree_fixpt_tmp_6; /* ufix3 */
    logic [2:0] decisionTree_fixpt_tmp_7; /* ufix3 */
    logic [2:0] decisionTree_fixpt_tmp_8; /* ufix3 */
    logic [2:0] decisionTree_fixpt_tmp_9; /* ufix3 */
    logic [2:0] decisionTree_fixpt_tmp_10; /* ufix3 */
    logic [2:0] decisionTree_fixpt_tmp_11; /* ufix3 */
    logic [2:0] decisionTree_fixpt_tmp_12; /* ufix3 */
    logic [2:0] decisionTree_fixpt_tmp_13; /* ufix3 */
    logic [2:0] decisionTree_fixpt_tmp_14; /* ufix3 */
    logic [2:0] decisionTree_fixpt_tmp_15; /* ufix3 */
    logic [2:0] decisionTree_fixpt_tmp_16; /* ufix3 */
    logic [2:0] decisionTree_fixpt_tmp_17; /* ufix3 */
    logic [2:0] decisionTree_fixpt_tmp_18; /* ufix3 */
```



```

logic [2:0] decisionTree_fixpt_tmp_19; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_20; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_21; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_22; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_23; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_24; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_25; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_26; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_27; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_28; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_29; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_30; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_31; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_32; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_33; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_34; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_35; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_36; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_37; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_38; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_39; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_40; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_41; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_42; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_43; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_44; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_45; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_46; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_47; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_48; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_49; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_50; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_51; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_52; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_53; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_54; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_55; /* ufix3 */

```

```

logic [2:0] decisionTree_fixpt_tmp_56; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_57; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_58; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_59; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_60; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_61; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_62; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_63; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_64; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_65; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_66; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_67; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_68; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_69; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_70; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_71; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_72; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_73; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_74; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_75; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_76; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_77; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_78; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_79; /* ufix3 */
logic [2:0] decisionTree_fixpt_tmp_80; /* ufix3 */

```

```

always_comb begin
    if (features[27] <= 13'b0110010000010) begin
        decisionTree_fixpt_tmp = 3'b010;
    end
    else begin
        decisionTree_fixpt_tmp = 3'b000;
    end
    if (features[3] <= 13'b0010000101000) begin
        decisionTree_fixpt_tmp = 3'b010;
    end
end

```

```

if (features[11] <= 13'b0010101000110) begin
    decisionTree_fixpt_tmp = 3'b010;
end
if (features[7] <= 13'b0001100111000) begin
    decisionTree_fixpt_tmp_0 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_0 = 3'b010;
end
if (features[112] > 13'b0010110101110) begin
    decisionTree_fixpt_tmp_0 = 3'b000;
end
if (features[98] <= 13'b0100100001000) begin
    decisionTree_fixpt_tmp_0 = decisionTree_fixpt_tmp;
end
if (features[61] <= 13'b0000111100001) begin
    decisionTree_fixpt_tmp_1 = 3'b001;
end
else begin
    decisionTree_fixpt_tmp_1 = 3'b000;
end
if (features[44] > 13'b0010011110100) begin
    decisionTree_fixpt_tmp_1 = 3'b100;
end
if (features[13] <= 13'b0001011010010) begin
    decisionTree_fixpt_tmp_2 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_2 = 3'b010;
end
if (features[67] <= 13'b0001100010000) begin
    decisionTree_fixpt_tmp_3 = 3'b010;
end
else begin
    decisionTree_fixpt_tmp_3 = 3'b000;
end
if (features[33] > 13'b0010111110010) begin

```

```

    decisionTree_fixpt_tmp_3 = 3'b010;
end
if (features[5] > 13'b0001110110000) begin
    decisionTree_fixpt_tmp_3 = 3'b100;
end
if (features[2] <= 13'b0001110100000) begin
    decisionTree_fixpt_tmp_3 = 3'b000;
end
if (features[100] <= 13'b0010100011100) begin
    decisionTree_fixpt_tmp_4 = 3'b010;
end
else begin
    decisionTree_fixpt_tmp_4 = 3'b000;
end
if (features[1] > 13'b0000111111011) begin
    decisionTree_fixpt_tmp_4 = 3'b000;
end
if (features[7] <= 13'b0001000100101) begin
    decisionTree_fixpt_tmp_4 = 3'b010;
end
if (features[10] <= 13'b0001010100100) begin
    decisionTree_fixpt_tmp_4 = decisionTree_fixpt_tmp_3;
end
if (features[22] > 13'b0100100001101) begin
    decisionTree_fixpt_tmp_4 = 3'b010;
end
if (features[5] <= 13'b0000000001001) begin
    decisionTree_fixpt_tmp_4 = decisionTree_fixpt_tmp_2;
end
if (features[12] <= 13'b0000001101000) begin
    decisionTree_fixpt_tmp_4 = decisionTree_fixpt_tmp_1;
end
if (features[1] <= 13'b0000110011010) begin
    decisionTree_fixpt_tmp_4 = decisionTree_fixpt_tmp_0;
end
if (features[32] <= 13'b0010111100111) begin
    decisionTree_fixpt_tmp_5 = 3'b001;
end

```

```

end
else begin
    decisionTree_fixpt_tmp_5 = 3'b000;
end
if (features[4] <= 13'b0000101101111) begin
    decisionTree_fixpt_tmp_5 = 3'b000;
end
if (features[1] <= 13'b0101001001000) begin
    decisionTree_fixpt_tmp_5 = 3'b000;
end
if (features[44] <= 13'b0001001000110) begin
    decisionTree_fixpt_tmp_5 = 3'b000;
end
if (features[5] <= 13'b0001100101000) begin
    decisionTree_fixpt_tmp_6 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_6 = 3'b001;
end
if (features[2] > 13'b0011010100000) begin
    decisionTree_fixpt_tmp_6 = 3'b000;
end
if (features[2] <= 13'b0010101011010) begin
    decisionTree_fixpt_tmp_6 = decisionTree_fixpt_tmp_5;
end
if (features[3] <= 13'b0001000101000) begin
    decisionTree_fixpt_tmp_7 = 3'b001;
end
else begin
    decisionTree_fixpt_tmp_7 = 3'b000;
end
if (features[84] > 13'b0000101110100) begin
    decisionTree_fixpt_tmp_7 = 3'b000;
end
if (features[79] <= 13'b0001010000100) begin
    decisionTree_fixpt_tmp_8 = 3'b100;
end

```

```

else begin
    decisionTree_fixpt_tmp_8 = 3'b000;
end
if (features[99] <= 13'b0001011110101) begin
    decisionTree_fixpt_tmp_8 = decisionTree_fixpt_tmp_7;
end
if (features[1] <= 13'b0101010010010) begin
    decisionTree_fixpt_tmp_9 = 3'b001;
end
else begin
    decisionTree_fixpt_tmp_9 = 3'b000;
end
if (features[73] <= 13'b0001001101110) begin
    decisionTree_fixpt_tmp_9 = 3'b100;
end
if (features[94] <= 13'b0001101000101) begin
    decisionTree_fixpt_tmp_9 = 3'b000;
end
if (features[40] <= 13'b0000111100110) begin
    decisionTree_fixpt_tmp_9 = decisionTree_fixpt_tmp_8;
end
if (features[59] <= 13'b0010011010100) begin
    decisionTree_fixpt_tmp_10 = 3'b001;
end
else begin
    decisionTree_fixpt_tmp_10 = 3'b000;
end
if (features[36] <= 13'b0011111011111) begin
    decisionTree_fixpt_tmp_10 = 3'b000;
end
if (features[32] <= 13'b0010101101111) begin
    decisionTree_fixpt_tmp_11 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_11 = 3'b010;
end
if (features[133] <= 13'b0000101011101) begin

```

```

    decisionTree_fixpt_tmp_11 = decisionTree_fixpt_tmp_10;
end
if (features[30] <= 13'b0010101111010) begin
    decisionTree_fixpt_tmp_12 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_12 = 3'b001;
end
if (features[164] <= 13'b0001110001010) begin
    decisionTree_fixpt_tmp_12 = decisionTree_fixpt_tmp_11;
end
if (features[31] <= 13'b0011010000110) begin
    decisionTree_fixpt_tmp_13 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_13 = 3'b001;
end
if (features[59] > 13'b0010010010100) begin
    decisionTree_fixpt_tmp_13 = 3'b000;
end
if (features[3] > 13'b0000010001101) begin
    decisionTree_fixpt_tmp_13 = 3'b000;
end
if (features[36] <= 13'b0100000010000) begin
    decisionTree_fixpt_tmp_13 = 3'b000;
end
if (features[186] > 13'b0011100011110) begin
    decisionTree_fixpt_tmp_13 = 3'b001;
end
if (features[90] <= 13'b0100011010110) begin
    decisionTree_fixpt_tmp_14 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_14 = 3'b001;
end
if (features[139] > 13'b0011011110000) begin
    decisionTree_fixpt_tmp_14 = 3'b001;
end

```

```

end
if (features[124] <= 13'b0011001011010) begin
    decisionTree_fixpt_tmp_14 = decisionTree_fixpt_tmp_13;
end
if (features[179] > 13'b0111000111010) begin
    decisionTree_fixpt_tmp_14 = 3'b001;
end
if (features[45] <= 13'b0001100111010) begin
    decisionTree_fixpt_tmp_15 = 3'b011;
end
else begin
    decisionTree_fixpt_tmp_15 = 3'b000;
end
if (features[61] <= 13'b0100001100110) begin
    decisionTree_fixpt_tmp_15 = 3'b000;
end
if (features[35] > 13'b0011100101001) begin
    decisionTree_fixpt_tmp_15 = 3'b001;
end
if (features[54] <= 13'b0100110101000) begin
    decisionTree_fixpt_tmp_15 = decisionTree_fixpt_tmp_14;
end
if (features[69] <= 13'b0010001110010) begin
    decisionTree_fixpt_tmp_16 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_16 = 3'b001;
end
if (features[2] > 13'b0001001101010) begin
    decisionTree_fixpt_tmp_16 = 3'b000;
end
if (features[4] <= 13'b0010001100000) begin
    decisionTree_fixpt_tmp_16 = decisionTree_fixpt_tmp_15;
end
if (features[7] <= 13'b0001111100011) begin
    decisionTree_fixpt_tmp_17 = 3'b010;
end

```



```

else begin
    decisionTree_fixpt_tmp_17 = 3'b000;
end
if (features[9] <= 13'b0010010011010) begin
    decisionTree_fixpt_tmp_17 = 3'b000;
end
if (features[143] > 13'b00001000110001) begin
    decisionTree_fixpt_tmp_17 = 3'b010;
end
if (features[2] <= 13'b0100110010110) begin
    decisionTree_fixpt_tmp_17 = decisionTree_fixpt_tmp_16;
end
if (features[3] <= 13'b00000001101110) begin
    decisionTree_fixpt_tmp_18 = 3'b001;
end
else begin
    decisionTree_fixpt_tmp_18 = 3'b000;
end
if (features[50] <= 13'b0100101111100) begin
    decisionTree_fixpt_tmp_18 = decisionTree_fixpt_tmp_17;
end
if (features[0] <= 13'b0110111001000) begin
    decisionTree_fixpt_tmp_18 = decisionTree_fixpt_tmp_12;
end
if (features[27] <= 13'b00000001111000) begin
    decisionTree_fixpt_tmp_18 = decisionTree_fixpt_tmp_9;
end
if (features[57] <= 13'b0010101011100) begin
    decisionTree_fixpt_tmp_19 = 3'b001;
end
else begin
    decisionTree_fixpt_tmp_19 = 3'b000;
end
if (features[5] <= 13'b00000101001110) begin
    decisionTree_fixpt_tmp_19 = 3'b000;
end
if (features[82] > 13'b0010111100101) begin

```

```

    decisionTree_fixpt_tmp_19 = 3'b000;
end
if (features[3] > 13'b00000011010010) begin
    decisionTree_fixpt_tmp_19 = 3'b000;
end
if (features[36] <= 13'b0100001011000) begin
    decisionTree_fixpt_tmp_19 = decisionTree_fixpt_tmp_18;
end
if (features[42] <= 13'b0010111100000) begin
    decisionTree_fixpt_tmp_20 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_20 = 3'b100;
end
if (features[26] <= 13'b00000100001000) begin
    decisionTree_fixpt_tmp_21 = 3'b100;
end
else begin
    decisionTree_fixpt_tmp_21 = 3'b000;
end
if (features[149] > 13'b00000001110010) begin
    decisionTree_fixpt_tmp_21 = 3'b010;
end
if (features[1] <= 13'b0011111111100) begin
    decisionTree_fixpt_tmp_21 = decisionTree_fixpt_tmp_20;
end
if (features[49] <= 13'b0001110111010) begin
    decisionTree_fixpt_tmp_22 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_22 = 3'b011;
end
if (features[8] > 13'b00000110011001) begin
    decisionTree_fixpt_tmp_22 = 3'b010;
end
if (features[40] <= 13'b00010100000001) begin
    decisionTree_fixpt_tmp_22 = 3'b000;
end

```

```

end
if (features[51] <= 13'b0010010001000) begin
    decisionTree_fixpt_tmp_23 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_23 = 3'b011;
end
if (features[11] <= 13'b0000101011100) begin
    decisionTree_fixpt_tmp_23 = 3'b000;
end
if (features[55] <= 13'b0100010101110) begin
    decisionTree_fixpt_tmp_23 = 3'b000;
end
if (features[60] <= 13'b0001111110110) begin
    decisionTree_fixpt_tmp_24 = 3'b011;
end
else begin
    decisionTree_fixpt_tmp_24 = 3'b000;
end
if (features[17] > 13'b0001010111010) begin
    decisionTree_fixpt_tmp_24 = 3'b000;
end
if (features[32] <= 13'b0010100000100) begin
    decisionTree_fixpt_tmp_24 = decisionTree_fixpt_tmp_23;
end
if (features[4] > 13'b0001100110110) begin
    decisionTree_fixpt_tmp_24 = 3'b000;
end
if (features[18] <= 13'b0000001110000) begin
    decisionTree_fixpt_tmp_24 = decisionTree_fixpt_tmp_22;
end
if (features[58] <= 13'b0010110001010) begin
    decisionTree_fixpt_tmp_25 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_25 = 3'b010;
end
end

```

```

if (features[136] <= 13'b0000110001111) begin
    decisionTree_fixpt_tmp_26 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_26 = 3'b010;
end
if (features[67] <= 13'b0100110110100) begin
    decisionTree_fixpt_tmp_27 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_27 = 3'b011;
end
if (features[152] <= 13'b0000000111000) begin
    decisionTree_fixpt_tmp_28 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_28 = 3'b010;
end
if (features[1] <= 13'b0110101110010) begin
    decisionTree_fixpt_tmp_29 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_29 = 3'b010;
end
if (features[5] <= 13'b0001000110110) begin
    decisionTree_fixpt_tmp_30 = 3'b011;
end
else begin
    decisionTree_fixpt_tmp_30 = 3'b010;
end
if (features[52] <= 13'b0001101100000) begin
    decisionTree_fixpt_tmp_30 = 3'b000;
end
if (features[18] > 13'b0000000010110) begin
    decisionTree_fixpt_tmp_30 = 3'b000;
end
if (features[5] <= 13'b0000010011010) begin

```

```

    decisionTree_fixpt_tmp_30 = decisionTree_fixpt_tmp_29;
end
if (features[99] <= 13'b0000100000111) begin
    decisionTree_fixpt_tmp_31 = 3'b100;
end
else begin
    decisionTree_fixpt_tmp_31 = 3'b000;
end
if (features[108] > 13'b0000001011110) begin
    decisionTree_fixpt_tmp_31 = 3'b000;
end
if (features[105] <= 13'b0001101000110) begin
    decisionTree_fixpt_tmp_31 = 3'b000;
end
if (features[1] <= 13'b0010101010010) begin
    decisionTree_fixpt_tmp_31 = 3'b100;
end
if (features[96] <= 13'b0000000000010) begin
    decisionTree_fixpt_tmp_31 = decisionTree_fixpt_tmp_30;
end
if (features[185] > 13'b0000000001111) begin
    decisionTree_fixpt_tmp_31 = 3'b001;
end
if (features[0] <= 13'b0110001101110) begin
    decisionTree_fixpt_tmp_31 = decisionTree_fixpt_tmp_28;
end
if (features[3] > 13'b0101010001010) begin
    decisionTree_fixpt_tmp_31 = 3'b100;
end
if (features[72] <= 13'b0000000000101) begin
    decisionTree_fixpt_tmp_31 = decisionTree_fixpt_tmp_27;
end
if (features[23] <= 13'b0000000011110) begin
    decisionTree_fixpt_tmp_31 = decisionTree_fixpt_tmp_26;
end
if (features[21] <= 13'b0000001110001) begin
    decisionTree_fixpt_tmp_32 = 3'b011;
end

```

```

end
else begin
    decisionTree_fixpt_tmp_32 = 3'b000;
end
if (features[1] <= 13'b0111111101111) begin
    decisionTree_fixpt_tmp_32 = decisionTree_fixpt_tmp_31;
end
if (features[157] > 13'b0100011101111) begin
    decisionTree_fixpt_tmp_32 = 3'b001;
end
if (features[4] <= 13'b0000000001100) begin
    decisionTree_fixpt_tmp_32 = decisionTree_fixpt_tmp_25;
end
if (features[5] <= 13'b0001100011100) begin
    decisionTree_fixpt_tmp_33 = 3'b100;
end
else begin
    decisionTree_fixpt_tmp_33 = 3'b000;
end
if (features[8] <= 13'b0000100001010) begin
    decisionTree_fixpt_tmp_33 = 3'b000;
end
if (features[14] <= 13'b0000101100010) begin
    decisionTree_fixpt_tmp_34 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_34 = 3'b010;
end
if (features[37] <= 13'b0000111100010) begin
    decisionTree_fixpt_tmp_34 = decisionTree_fixpt_tmp_33;
end
if (features[123] > 13'b0001010001000) begin
    decisionTree_fixpt_tmp_34 = 3'b010;
end
if (features[2] <= 13'b0010011011001) begin
    decisionTree_fixpt_tmp_35 = 3'b000;
end

```

```

else begin
    decisionTree_fixpt_tmp_35 = 3'b100;
end
if (features[37] <= 13'b0000111010110) begin
    decisionTree_fixpt_tmp_36 = 3'b100;
end
else begin
    decisionTree_fixpt_tmp_36 = 3'b010;
end
if (features[4] > 13'b0001101011010) begin
    decisionTree_fixpt_tmp_36 = 3'b000;
end
if (features[59] <= 13'b0001011000000) begin
    decisionTree_fixpt_tmp_37 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_37 = 3'b010;
end
if (features[19] > 13'b0000000110110) begin
    decisionTree_fixpt_tmp_37 = 3'b000;
end
if (features[2] <= 13'b0011110001011) begin
    decisionTree_fixpt_tmp_38 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_38 = 3'b100;
end
if (features[1] > 13'b0110110010110) begin
    decisionTree_fixpt_tmp_38 = 3'b000;
end
if (features[2] <= 13'b0101000001010) begin
    decisionTree_fixpt_tmp_39 = 3'b100;
end
else begin
    decisionTree_fixpt_tmp_39 = 3'b000;
end
if (features[3] <= 13'b0010101110101) begin

```

```

    decisionTree_fixpt_tmp_39 = decisionTree_fixpt_tmp_38;
end
if (features[6] <= 13'b0001000001100) begin
    decisionTree_fixpt_tmp_39 = decisionTree_fixpt_tmp_37;
end
if (features[25] <= 13'b00000001101010) begin
    decisionTree_fixpt_tmp_39 = decisionTree_fixpt_tmp_36;
end
if (features[4] > 13'b0010001100111) begin
    decisionTree_fixpt_tmp_39 = 3'b000;
end
if (features[37] <= 13'b0011000011101) begin
    decisionTree_fixpt_tmp_40 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_40 = 3'b010;
end
if (features[26] <= 13'b0000111010110) begin
    decisionTree_fixpt_tmp_40 = 3'b010;
end
if (features[67] <= 13'b0010011111000) begin
    decisionTree_fixpt_tmp_40 = decisionTree_fixpt_tmp_39;
end
if (features[7] <= 13'b0000000001100) begin
    decisionTree_fixpt_tmp_41 = 3'b010;
end
else begin
    decisionTree_fixpt_tmp_41 = 3'b000;
end
if (features[4] <= 13'b0000001110101) begin
    decisionTree_fixpt_tmp_42 = 3'b010;
end
else begin
    decisionTree_fixpt_tmp_42 = 3'b000;
end
if (features[86] <= 13'b0110010111110) begin
    decisionTree_fixpt_tmp_42 = decisionTree_fixpt_tmp_41;

```



```

end
if (features[26] <= 13'b0001110000000) begin
    decisionTree_fixpt_tmp_42 = decisionTree_fixpt_tmp_40;
end
if (features[1] <= 13'b0011110001010) begin
    decisionTree_fixpt_tmp_42 = decisionTree_fixpt_tmp_35;
end
if (features[22] <= 13'b0000001110010) begin
    decisionTree_fixpt_tmp_42 = decisionTree_fixpt_tmp_34;
end
if (features[98] <= 13'b0001011110010) begin
    decisionTree_fixpt_tmp_42 = decisionTree_fixpt_tmp_32;
end
if (features[81] <= 13'b0000000000100) begin
    decisionTree_fixpt_tmp_42 = decisionTree_fixpt_tmp_24;
end
if (features[0] <= 13'b0101111010110) begin
    decisionTree_fixpt_tmp_42 = decisionTree_fixpt_tmp_21;
end
if (features[3] <= 13'b0001101111110) begin
    decisionTree_fixpt_tmp_42 = decisionTree_fixpt_tmp_19;
end
if (features[59] <= 13'b0000000000100) begin
    decisionTree_fixpt_tmp_42 = decisionTree_fixpt_tmp_6;
end
if (features[1] <= 13'b0110110100100) begin
    decisionTree_fixpt_tmp_43 = 3'b100;
end
else begin
    decisionTree_fixpt_tmp_43 = 3'b000;
end
if (features[55] > 13'b0001010100011) begin
    decisionTree_fixpt_tmp_43 = 3'b010;
end
if (features[95] <= 13'b0000001000000) begin
    decisionTree_fixpt_tmp_44 = 3'b011;
end

```

```

else begin
    decisionTree_fixpt_tmp_44 = 3'b000;
end
if (features[1] <= 13'b0111001110110) begin
    decisionTree_fixpt_tmp_45 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_45 = 3'b010;
end
if (features[9] <= 13'b0000101110111) begin
    decisionTree_fixpt_tmp_45 = decisionTree_fixpt_tmp_44;
end
if (features[27] <= 13'b0000110010010) begin
    decisionTree_fixpt_tmp_46 = 3'b010;
end
else begin
    decisionTree_fixpt_tmp_46 = 3'b000;
end
if (features[56] <= 13'b0001011100111) begin
    decisionTree_fixpt_tmp_46 = 3'b000;
end
if (features[6] > 13'b0010111001000) begin
    decisionTree_fixpt_tmp_46 = 3'b100;
end
if (features[23] <= 13'b0000001010000) begin
    decisionTree_fixpt_tmp_47 = 3'b010;
end
else begin
    decisionTree_fixpt_tmp_47 = 3'b000;
end
if (features[33] <= 13'b0000110001100) begin
    decisionTree_fixpt_tmp_47 = decisionTree_fixpt_tmp_46;
end
if (features[0] <= 13'b0111011100110) begin
    decisionTree_fixpt_tmp_48 = 3'b011;
end
else begin

```

```

    decisionTree_fixpt_tmp_48 = 3'b000;
end
if (features[56] <= 13'b0100101110010) begin
    decisionTree_fixpt_tmp_48 = decisionTree_fixpt_tmp_47;
end
if (features[18] <= 13'b00000001100100) begin
    decisionTree_fixpt_tmp_48 = decisionTree_fixpt_tmp_45;
end
if (features[25] <= 13'b00000001110110) begin
    decisionTree_fixpt_tmp_48 = decisionTree_fixpt_tmp_43;
end
if (features[2] <= 13'b0101110010110) begin
    decisionTree_fixpt_tmp_48 = decisionTree_fixpt_tmp_42;
end
if (features[0] <= 13'b0100101010001) begin
    decisionTree_fixpt_tmp_48 = decisionTree_fixpt_tmp_4;
end
if (features[11] <= 13'b0011001111001) begin
    decisionTree_fixpt_tmp_49 = 3'b010;
end
else begin
    decisionTree_fixpt_tmp_49 = 3'b000;
end
if (features[16] <= 13'b0001110011000) begin
    decisionTree_fixpt_tmp_49 = 3'b000;
end
if (features[31] <= 13'b0111010110000) begin
    decisionTree_fixpt_tmp_50 = 3'b100;
end
else begin
    decisionTree_fixpt_tmp_50 = 3'b010;
end
if (features[1] > 13'b0110001000110) begin
    decisionTree_fixpt_tmp_50 = 3'b010;
end
if (features[57] <= 13'b0010100111010) begin
    decisionTree_fixpt_tmp_51 = 3'b000;
end

```

```

end
else begin
    decisionTree_fixpt_tmp_51 = 3'b010;
end
if (features[138] <= 13'b00000001001000) begin
    decisionTree_fixpt_tmp_51 = decisionTree_fixpt_tmp_50;
end
if (features[4] <= 13'b00000111111100) begin
    decisionTree_fixpt_tmp_51 = decisionTree_fixpt_tmp_49;
end
if (features[87] <= 13'b0010000110010) begin
    decisionTree_fixpt_tmp_52 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_52 = 3'b001;
end
if (features[71] <= 13'b0011011010110) begin
    decisionTree_fixpt_tmp_53 = 3'b010;
end
else begin
    decisionTree_fixpt_tmp_53 = 3'b000;
end
if (features[33] <= 13'b0100111111110) begin
    decisionTree_fixpt_tmp_53 = decisionTree_fixpt_tmp_52;
end
if (features[52] <= 13'b0011011000100) begin
    decisionTree_fixpt_tmp_54 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_54 = 3'b010;
end
if (features[30] <= 13'b0100010001111) begin
    decisionTree_fixpt_tmp_55 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_55 = 3'b001;
end

```

```

if (features[43] <= 13'b0011111011010) begin
    decisionTree_fixpt_tmp_55 = 3'b001;
end
if (features[27] <= 13'b0010110100110) begin
    decisionTree_fixpt_tmp_55 = 3'b000;
end
if (features[86] <= 13'b0010000011101) begin
    decisionTree_fixpt_tmp_55 = decisionTree_fixpt_tmp_54;
end
if (features[1] <= 13'b0101100001100) begin
    decisionTree_fixpt_tmp_55 = decisionTree_fixpt_tmp_53;
end
if (features[1] <= 13'b0110111001110) begin
    decisionTree_fixpt_tmp_56 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_56 = 3'b001;
end
if (features[4] <= 13'b0000100011111) begin
    decisionTree_fixpt_tmp_56 = 3'b010;
end
if (features[10] > 13'b0010101001000) begin
    decisionTree_fixpt_tmp_56 = 3'b000;
end
if (features[0] <= 13'b0000000010100) begin
    decisionTree_fixpt_tmp_56 = 3'b010;
end
if (features[3] > 13'b0001100011110) begin
    decisionTree_fixpt_tmp_56 = 3'b000;
end
if (features[10] <= 13'b0010001111010) begin
    decisionTree_fixpt_tmp_56 = decisionTree_fixpt_tmp_55;
end
if (features[0] <= 13'b0110110001100) begin
    decisionTree_fixpt_tmp_57 = 3'b010;
end
else begin

```

```

    decisionTree_fixpt_tmp_57 = 3'b000;
end
if (features[12] <= 13'b0101111100010) begin
    decisionTree_fixpt_tmp_58 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_58 = 3'b010;
end
if (features[48] <= 13'b0110110000000) begin
    decisionTree_fixpt_tmp_58 = decisionTree_fixpt_tmp_57;
end
if (features[16] <= 13'b0100100011000) begin
    decisionTree_fixpt_tmp_58 = decisionTree_fixpt_tmp_56;
end
if (features[34] <= 13'b0100101110110) begin
    decisionTree_fixpt_tmp_59 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_59 = 3'b010;
end
if (features[1] <= 13'b0110101000100) begin
    decisionTree_fixpt_tmp_60 = 3'b100;
end
else begin
    decisionTree_fixpt_tmp_60 = 3'b000;
end
if (features[4] <= 13'b0001100010100) begin
    decisionTree_fixpt_tmp_60 = decisionTree_fixpt_tmp_59;
end
if (features[41] <= 13'b0101000000100) begin
    decisionTree_fixpt_tmp_61 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_61 = 3'b010;
end
if (features[0] <= 13'b011111111001) begin
    decisionTree_fixpt_tmp_61 = 3'b000;
end

```

```

end
if (features[8] <= 13'b0001100111011) begin
    decisionTree_fixpt_tmp_61 = decisionTree_fixpt_tmp_60;
end
if (features[2] <= 13'b0011110010010) begin
    decisionTree_fixpt_tmp_61 = decisionTree_fixpt_tmp_58;
end
if (features[8] <= 13'b0000101101110) begin
    decisionTree_fixpt_tmp_61 = decisionTree_fixpt_tmp_51;
end
if (features[36] <= 13'b0100011010000) begin
    decisionTree_fixpt_tmp_61 = decisionTree_fixpt_tmp_48;
end
if (features[29] <= 13'b0010100100110) begin
    decisionTree_fixpt_tmp_62 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_62 = 3'b010;
end
if (features[50] <= 13'b0001000011010) begin
    decisionTree_fixpt_tmp_63 = 3'b001;
end
else begin
    decisionTree_fixpt_tmp_63 = 3'b100;
end
if (features[2] <= 13'b0011010101000) begin
    decisionTree_fixpt_tmp_63 = decisionTree_fixpt_tmp_62;
end
if (features[59] <= 13'b0000010001010) begin
    decisionTree_fixpt_tmp_64 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_64 = 3'b010;
end
if (features[26] > 13'b0001010001010) begin
    decisionTree_fixpt_tmp_64 = 3'b000;
end

```

```

if (features[18] <= 13'b00000010111110) begin
    decisionTree_fixpt_tmp_65 = 3'b010;
end
else begin
    decisionTree_fixpt_tmp_65 = 3'b000;
end
if (features[83] <= 13'b00000000001100) begin
    decisionTree_fixpt_tmp_65 = decisionTree_fixpt_tmp_64;
end
if (features[40] > 13'b0100101101100) begin
    decisionTree_fixpt_tmp_65 = 3'b100;
end
if (features[43] <= 13'b0100000100110) begin
    decisionTree_fixpt_tmp_66 = 3'b010;
end
else begin
    decisionTree_fixpt_tmp_66 = 3'b100;
end
if (features[58] > 13'b0100110011010) begin
    decisionTree_fixpt_tmp_66 = 3'b011;
end
if (features[2] <= 13'b0110111000100) begin
    decisionTree_fixpt_tmp_66 = decisionTree_fixpt_tmp_65;
end
if (features[130] > 13'b0001010111110) begin
    decisionTree_fixpt_tmp_66 = 3'b010;
end
if (features[3] <= 13'b0100000110011) begin
    decisionTree_fixpt_tmp_67 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_67 = 3'b010;
end
if (features[44] <= 13'b0011011000010) begin
    decisionTree_fixpt_tmp_68 = 3'b000;
end
else begin

```



```

    decisionTree_fixpt_tmp_68 = 3'b100;
end
if (features[26] <= 13'b0010001011101) begin
    decisionTree_fixpt_tmp_68 = decisionTree_fixpt_tmp_67;
end
if (features[7] <= 13'b0010110001000) begin
    decisionTree_fixpt_tmp_68 = decisionTree_fixpt_tmp_66;
end
if (features[1] <= 13'b0101101010010) begin
    decisionTree_fixpt_tmp_68 = decisionTree_fixpt_tmp_63;
end
if (features[1] <= 13'b0110011111101) begin
    decisionTree_fixpt_tmp_69 = 3'b100;
end
else begin
    decisionTree_fixpt_tmp_69 = 3'b000;
end
if (features[84] > 13'b0001110011010) begin
    decisionTree_fixpt_tmp_69 = 3'b010;
end
if (features[22] <= 13'b00000001100110) begin
    decisionTree_fixpt_tmp_70 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_70 = 3'b100;
end
if (features[33] <= 13'b00000001001000) begin
    decisionTree_fixpt_tmp_70 = 3'b000;
end
if (features[45] <= 13'b0100001011000) begin
    decisionTree_fixpt_tmp_71 = 3'b010;
end
else begin
    decisionTree_fixpt_tmp_71 = 3'b100;
end
if (features[130] <= 13'b00000000110001) begin
    decisionTree_fixpt_tmp_71 = decisionTree_fixpt_tmp_70;
end

```

```

end
if (features[26] <= 13'b0000010111100) begin
    decisionTree_fixpt_tmp_71 = decisionTree_fixpt_tmp_69;
end
if (features[102] <= 13'b0010110000000) begin
    decisionTree_fixpt_tmp_71 = decisionTree_fixpt_tmp_68;
end
if (features[57] <= 13'b0010000001010) begin
    decisionTree_fixpt_tmp_72 = 3'b100;
end
else begin
    decisionTree_fixpt_tmp_72 = 3'b010;
end
if (features[58] <= 13'b0000000010000) begin
    decisionTree_fixpt_tmp_73 = 3'b100;
end
else begin
    decisionTree_fixpt_tmp_73 = 3'b000;
end
if (features[6] <= 13'b0011010100100) begin
    decisionTree_fixpt_tmp_74 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_74 = 3'b100;
end
if (features[55] <= 13'b0010000000010) begin
    decisionTree_fixpt_tmp_74 = decisionTree_fixpt_tmp_73;
end
if (features[23] <= 13'b0001010100000) begin
    decisionTree_fixpt_tmp_74 = decisionTree_fixpt_tmp_72;
end
if (features[99] <= 13'b0001011101111) begin
    decisionTree_fixpt_tmp_75 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_75 = 3'b100;
end

```

```

if (features[26] <= 13'b0011110011101) begin
    decisionTree_fixpt_tmp_76 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_76 = 3'b010;
end
if (features[99] <= 13'b0100001101000) begin
    decisionTree_fixpt_tmp_77 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_77 = 3'b100;
end
if (features[2] <= 13'b0011110110110) begin
    decisionTree_fixpt_tmp_77 = decisionTree_fixpt_tmp_76;
end
if (features[107] > 13'b0010101111110) begin
    decisionTree_fixpt_tmp_77 = 3'b000;
end
if (features[1] <= 13'b0100010100010) begin
    decisionTree_fixpt_tmp_78 = 3'b100;
end
else begin
    decisionTree_fixpt_tmp_78 = 3'b000;
end
if (features[0] <= 13'b0111111101100) begin
    decisionTree_fixpt_tmp_78 = decisionTree_fixpt_tmp_77;
end
if (features[19] <= 13'b0110011000010) begin
    decisionTree_fixpt_tmp_79 = 3'b000;
end
else begin
    decisionTree_fixpt_tmp_79 = 3'b010;
end
if (features[3] > 13'b0010011111000) begin
    decisionTree_fixpt_tmp_79 = 3'b000;
end
if (features[64] > 13'b0111110000100) begin

```

```

    decisionTree_fixpt_tmp_79 = 3'b010;
end
if (features[13] <= 13'b0010100011110) begin
    decisionTree_fixpt_tmp_79 = 3'b100;
end
if (features[8] <= 13'b0010010111110) begin
    decisionTree_fixpt_tmp_79 = 3'b100;
end
if (features[41] <= 13'b0100010010000) begin
    decisionTree_fixpt_tmp_79 = decisionTree_fixpt_tmp_78;
end
if (features[0] <= 13'b00000001001111) begin
    decisionTree_fixpt_tmp_80 = 3'b010;
end
else begin
    decisionTree_fixpt_tmp_80 = 3'b000;
end
if (features[39] > 13'b0101110110110) begin
    decisionTree_fixpt_tmp_80 = 3'b000;
end
if (features[24] <= 13'b0111101110110) begin
    decisionTree_fixpt_tmp_80 = decisionTree_fixpt_tmp_79;
end
if (features[15] <= 13'b0010011001100) begin
    decisionTree_fixpt_tmp_80 = decisionTree_fixpt_tmp_75;
end
if (features[10] <= 13'b0001110100100) begin
    decisionTree_fixpt_tmp_80 = 3'b100;
end
if (features[3] <= 13'b0100000011010) begin
    output_rsvd_1 = 3'b001;
end
else begin
    output_rsvd_1 = 3'b000;
end
if (features[2] <= 13'b0010000001101) begin
    output_rsvd_1 = 3'b010;
end

```

```

end
if (features[21] <= 13'b0111101000000) begin
    output_rsvd_1 = decisionTree_fixpt_tmp_80;
end
if (features[33] <= 13'b0010000110010) begin
    output_rsvd_1 = decisionTree_fixpt_tmp_74;
end
if (features[12] <= 13'b0010010111110) begin
    output_rsvd_1 = decisionTree_fixpt_tmp_71;
end

if (features[4] <= 13'b0011001001010) begin
    output_rsvd_1 = decisionTree_fixpt_tmp_61;
end
end

assign output_rsvd = output_rsvd_1;

endmodule // decisionTree_fixpt

```

6.2 Parallel Preprocessing module :-

```

module ECGPreprocess #(
    parameter SIGNAL_LENGTH = 300,
    parameter DATA_WIDTH   = 13,
    parameter HEIGHT_TH     = 13'd1000,
    parameter OUTPUT_LENGTH = 187
)(
    input logic [DATA_WIDTH-1:0] ecg_signal [0:SIGNAL_LENGTH-1],
    output logic [DATA_WIDTH-1:0] processed_signal [0:OUTPUT_LENGTH-1]
);
    integer i, j;
    integer peak_count;
    integer peaks[0:SIGNAL_LENGTH-1];
    logic [12:0] min_value , max_value, range;
    logic [25:0] scaled_value;
    always@(*) begin
        peak_count = 0;
    end

```

```

    for (i = 0; i < OUTPUT_LENGTH; i = i + 1)
        processed_signal[i] = '0;
    for (i = 0; i < SIGNAL_LENGTH-1; i = i + 1) begin
        if (i-1>=0 && ecg_signal[i] > ecg_signal[i-1] &&
            ecg_signal[i] > ecg_signal[i+1] &&
            ecg_signal[i] >= HEIGHT_TH) begin
            peaks[peak_count] = i;
            peak_count = peak_count + 1;

        end
    end

    min_value = 13'h1FFF;
    max_value = 13'd0;
    if (peak_count >= 2) begin
        for (i = 0; i < OUTPUT_LENGTH ; i = i + 1) begin
            if((peaks[0] + i) <= (peaks[1] + 40) &&(peaks[0] + i) <
SIGNAL_LENGTH) begin
                processed_signal[i] = ecg_signal[peaks[0] + i];
                if (processed_signal[i] > max_value) max_value = processed_signal[i];
                if(processed_signal[i] < min_value) min_value = processed_signal[i];
            end
        end
        range = max_value - min_value;
        if (range == 0) range = 1;
        for (i = 0; i < OUTPUT_LENGTH; i = i + 1) begin
            if (processed_signal[i] > 0) begin
                scaled_value = (processed_signal[i] - min_value) * 13'd4095;
                processed_signal[i] = scaled_value / range;
            end
        end
    end
end

endmodule

```

6.3 Serial Preprocessing module :-

```

module ECGPreprocess #(
    parameter SIGNAL_LENGTH = 300,
    parameter DATA_WIDTH   = 13,
    parameter HEIGHT_TH     = 13'd2000,
    parameter OUTPUT_LENGTH = 187
)(
    input logic clk,

```

```

input logic rst,
input logic start,
input logic [DATA_WIDTH-1:0] ecg_signal [0:SIGNAL_LENGTH-1],
output logic [DATA_WIDTH-1:0] processed_signal [0:OUTPUT_LENGTH-1],
output logic done
);

typedef enum logic [1:0] {
    IDLE,
    FIND_PEAKS,
    PROCESS_SIGNAL,
    FINALIZE
} state_t;

state_t state, next_state;

integer i, peak_count;
integer peaks[0:SIGNAL_LENGTH-1];
integer current_index, idx;
logic [DATA_WIDTH-1:0] temp_signal [0:OUTPUT_LENGTH-1];
logic j;
logic [25:0] scaled_value;
logic [DATA_WIDTH-1:0] copy [0:SIGNAL_LENGTH-1];
logic [DATA_WIDTH-1:0] tempo [0:OUTPUT_LENGTH-1];
logic [12:0] min_value, max_value, range;
integer valid_length;
integer k;

always_ff @(posedge clk or posedge rst) begin
    if (rst) begin
        state <= IDLE;
    end else begin
        state <= next_state;
    end
end

always_comb begin
    next_state = state;
    case (state)
        IDLE: if (start) next_state = FIND_PEAKS;
        FIND_PEAKS: if (current_index >= SIGNAL_LENGTH - 1) next_state =
PROCESS_SIGNAL;

```

```

        PROCESS_SIGNAL: if (current_index >= OUTPUT_LENGTH) next_state =
FINALIZE;
        FINALIZE: if (idx >= OUTPUT_LENGTH) next_state = IDLE;
    endcase
end

always_ff @(posedge clk or posedge rst) begin
    if (rst) begin
        peak_count = 0;
        current_index = 0;
        j = 0;
        idx = 0;
        min_value = 13'h1FFF;
        max_value = 13'd0;
        for (i = 0; i < OUTPUT_LENGTH; i = i + 1) begin
            processed_signal[i] = '0;
            temp_signal[i] = '0;
        end
    end else begin
        case (state)
            IDLE: begin
                peak_count = 0;
                current_index = 0;
                idx = 0;
                j = 0;
                min_value = 13'h1FFF;
                max_value = 13'd0;
                if (start) begin
                    for (i = 0; i < SIGNAL_LENGTH; i = i + 1) begin
                        copy[i] = ecg_signal[i];
                    end
                end
            end
        end
    end

    FIND_PEAKS: begin
        if (current_index < SIGNAL_LENGTH - 1) begin
            if (current_index > 0 &&
                copy[current_index] > copy[current_index - 1] &&
                copy[current_index] > copy[current_index + 1] &&
                copy[current_index] >= HEIGHT_TH) begin
                peaks[peak_count] = current_index;
                peak_count = peak_count + 1;
            end
        end
    end
end

```



```

        current_index = current_index + 1;
    end else begin
        current_index = 0;
    end
end

PROCESS_SIGNAL: begin
    if (peak_count >= 2 && current_index < OUTPUT_LENGTH) begin
        if ((peaks[0] + current_index) <= (peaks[1] + 40) &&
            (peaks[0] + current_index) < SIGNAL_LENGTH) begin
            processed_signal[current_index] = copy[peaks[0] + current_index];
        end else begin
            processed_signal[current_index] <= '0';
        end
        if (processed_signal[current_index] > max_value) max_value =
processed_signal[current_index];
        if (processed_signal[current_index] < min_value) min_value =
processed_signal[current_index];
        current_index = current_index + 1;
    end else begin
        current_index = current_index + 1;
    end
end

FINALIZE: begin
    if (idx < OUTPUT_LENGTH) begin
        scaled_value = ((processed_signal[idx] - min_value) * 13'd4095) /
(max_value == min_value ? 1 : max_value - min_value);
        processed_signal[idx] = scaled_value;
        idx <= idx + 1;
    end
end
endcase
end
end

assign done = (state == FINALIZE);

endmodule

```

6.4 Clock Divider :-

```

module converter(
    input bigClk,
    output reg smallClk
);
    reg [15:0] count;
    reg temp = 0;
    initial begin
        smallClk = 0;
    end
    always @(posedge bigClk) begin
        if (temp == 0) begin
            smallClk = 0;
            temp = 1;
        end
        if (count < 8000) begin
            count = count+1;
        end else begin
            smallClk = ~smallClk;
            count = 0;
        end
    end
endmodule

module DFF(
    input clk,
    input reset,
    input d,
    output reg q
);
    always @(posedge clk) begin
        if(reset)begin
            q <= 0; // Use non-blocking assignment
        end
        else begin
            q <= d;
        end
    end
endmodule

```

6.5 Seven Segment decoder (Same for all) :-

```

module sevensegdecoder(

    input [3:0] nIn,

```

```

        output reg [6:0] ssOut
    );

    always @(nIn)
    case (nIn)
        4'h0: ssOut = 7'b1000000;
        4'h1: ssOut = 7'b1111001;
        4'h2: ssOut = 7'b0100100;
        4'h3: ssOut = 7'b0110000;
        4'h4: ssOut = 7'b0011001;
        4'h5: ssOut = 7'b0010010;
        4'h6: ssOut = 7'b0000010;
        4'h7: ssOut = 7'b1111000;
        4'h8: ssOut = 7'b0000000;
        4'h9: ssOut = 7'b0011000;
        4'hA: ssOut = 7'b0001000;
        4'hB: ssOut = 7'b0000011;
        4'hC: ssOut = 7'b1000110;
        4'hD: ssOut = 7'b0100001;
        4'hE: ssOut = 7'b0000110;
        4'hF: ssOut = 7'b0001110;
        default: ssOut = 7'b1001001;
    endcase

endmodule

```

6.6 Data Queue (same for all) :-

```

module DFF(
    input clk,
    input reset,
    input d,
    output reg q
);
    always @(posedge clk) begin
        if(reset)begin
            q <= 0; // Use non-blocking assignment
        end
        else begin
            q <= d;
        end
    end
end

```

```

endmodule

module shiftRegister#(
    parameter length = 300
)(
    input si,
    input clk,
    input rst,
    output [length-1:0] out
);
    genvar i;
    generate
        for (i = 0; i < length; i = i + 1) begin : shift
            if (i == 0) begin
                DFF dff_inst (
                    .clk(clk),
                    .reset(rst),
                    .d(si),
                    .q(out[i])
                );
            end
            else begin
                DFF dff_inst (
                    .clk(clk),
                    .reset(rst),
                    .d(out[i-1]),
                    .q(out[i])
                );
            end
        end
    endgenerate
endmodule

module queue #(
    parameter QUEUE_SIZE = 300, // Number of stages in the shift register
    parameter DATA_WIDTH = 13  // Width of the serial data word
)(
    input logic clk,
    input logic rst,
    input logic write_enable, // Add write enable signal
    input logic [DATA_WIDTH-1:0] serialData,
    // Declared as logic so that we can drive it with continuous assignments
    output [DATA_WIDTH-1:0] data [0:QUEUE_SIZE-1]

```

```

);

// Each bit of the serial data is shifted in its own chain.
// temp[j] is a QUEUE_SIZE-bit vector corresponding to bit j's chain.
wire [QUEUE_SIZE-1:0] temp [DATA_WIDTH-1:0];

genvar i;
generate
  for (i = 0; i < DATA_WIDTH; i = i + 1) begin : shift_registers
    shiftRegister #(
      .length(QUEUE_SIZE)
    ) shift_reg_inst (
      .si(write_enable ? serialData[i] : 1'b0), // Use write enable signal
      .clk(clk),
      .rst(rst),
      .out(temp[i])
    );
  end
endgenerate

genvar ii, jj;
generate
  for (ii = 0; ii < QUEUE_SIZE; ii = ii + 1) begin : data_words
    for (jj = 0; jj < DATA_WIDTH; jj = jj + 1) begin : bit_assign
      assign data[ii][jj] = temp[jj][ii];
    end
  end
endgenerate

endmodule

```

6.6 Top Module (serial Processing) :-

```

module top(
  input logic slowClk,
  input logic fastClk,
  input logic rst,
  input [12:0] serialData,
  input logic write_enable,
  output logic [2:0] dout
);

// Instantiate the queue module
wire [12:0] qout [0:299];

```

```

queue #(
    .QUEUE_SIZE(300),
    .DATA_WIDTH(13)
) q (
    .clk(slowClk), // Use slowClk for the queue
    .rst(rst),
    .write_enable(write_enable), // Connect write enable signal
    .serialData(serialData),
    .data(qout)
);

// Instantiate the ECGPreprocess module
wire [12:0] ppout [0:186];
logic preprocess_done;
ECGPreprocess #(
    .SIGNAL_LENGTH(300),
    .DATA_WIDTH(13),
    .HEIGHT_TH(13'd2500),
    .OUTPUT_LENGTH(187)
) pp (
    .clk(fastClk), // Use fastClk for preprocessing
    .rst(rst),
    .start(1'b1),
    .ecg_signal(qout),
    .processed_signal(ppout),
    .done(preprocess_done)
);

// Instantiate the decision tree module
decisionTree_fixpt myTree (
    .features(ppout),
    .output_rsvd(dout)
);

// Control write enable signal

```

```
endmodule
```

6.7 Top Module (Parallel Processing):-

```

module top(
    input logic    clk,
    input logic    rst,
    input logic [12:0] serialData,

```

```

        output logic [2:0] dout
    );

    wire [12:0] qout [0:299];
    queue #(
        .QUEUE_SIZE(300),
        .DATA_WIDTH(13)
    ) q (
        .clk(clk),
        .rst(rst),
        .serialData(serialData),
        .data(qout)
    );

    // Instantiate the ECGPreprocess module (assumed defined elsewhere)
    wire [12:0] ppout [0:186];
    ECGPreprocess #(
        .SIGNAL_LENGTH(300),
        .DATA_WIDTH(13),
        .HEIGHT_TH(13'd2500),
        .OUTPUT_LENGTH(187)
    ) pp (
        .ecg_signal(qout),
        .processed_signal(ppout)
    );

    // Instantiate the decision tree module (assumed defined elsewhere)
    decisionTree_fixpt myTree (
        .features(ppout),
        .output_rsvd(dout)
    );

Endmodule

```

TestBenches :-

1.Serial Preprocessing :-

```
module tb();
    reg fastClk, slowClk, rst;
    reg [12:0] serialData;
    wire [2:0] out;
    reg writeEn;
    logic [12:0] segment [1428] = '{13'b0001011011100, 13'b0001001100011,
13'b0001010010001, 13'b0001001111011, 13'b0001010010000, 13'b0001010000001,
13'b0001010010000, 13'b0001010001100, 13'b0001010011110, 13'b0001010011110,
13'b0001010110100, 13'b0001010111100, 13'b0001011001001, 13'b0001010111001,
13'b0001010111001, 13'b0001010100110,...(Array cropped due to large size) }';

    // Instantiate the top module
    top myTop(
        .slowClk(slowClk),
        .fastClk(fastClk),
        .rst(rst),
        .serialData(serialData),
        .dout(out),
        .write_enable(writeEn)
    );

    // Generate slow clock (e.g., 10ns period)
    initial begin
        slowClk = 0;
        forever #1 slowClk = ~slowClk; // 10ns clock period
    end

    // Generate fast clock (e.g., 1ns period)

    integer i;
    initial begin
        fastClk = 0;
        rst = 1;
        serialData = 13'b0;
        writeEn = 0;
```



```

#20 rst = 0;

// Write first 300 samples to the queue
writeEn = 1;
for (i = 0; i < 300; i = i + 1) begin
    @(posedge slowClk);
    serialData = segment[i];
end

// Disable queue writing
writeEn = 0;

// Run fast clock for 600 cycles
for(i=0; i < 600; i = i+1) begin
    fastClk = ~fastClk; #1;
end

// Enable queue writing and write the next 300 samples
writeEn = 1;
for (i = 300; i < 600; i = i + 1) begin
    @(posedge slowClk); // Wait for the positive edge of the slow clock
    serialData = segment[i]; // Input the next data sample
end

// Disable queue writing
writeEn = 0;

// Run fast clock for 600 cycles

// Enable queue writing and write the next 300 samples
writeEn = 1;
for (i = 600; i < 900; i = i + 1) begin
    @(posedge slowClk); // Wait for the positive edge of the slow clock
    serialData = segment[i]; // Input the next data sample
end

// Finish simulation
#1000 $finish;
end

endmodule

```

2.Parallel Preprocessing :-

```
module tb();
    reg clk, rst;
    reg [12:0] serialData;
    wire [2:0] out;
    top myTop(clk, rst, serialData, out);

    logic [12:0] segment [0:186] = '{13'b0111101101110, 13'b0100001001100,
    13'b0000101001000, 13'b0001010000100, 13'b0001001010010, 13'b0001011010100,
    13'b0001011011110, 13'b0001010110110, ...(array cropped due to large size)};

    initial begin
        clk = 0;
        forever #1 clk = ~clk; // 10ns clock period
    end

    integer i;
    initial begin
        rst = 1;
        serialData = 13'b0;
        #20 rst = 0;

        // Feed data samples at each clock cycle
        for (i = 0; i < 1428; i = i + 1) begin
            @(posedge clk); // Wait for the positive edge of the clock
            serialData = segment[i]; // Input the next data sample
        end

        #10 $finish;
    end
endmodule
```

Chapter 7: References

- [1] C. Chen, B. da Silva, C. Yang, C. Ma, J. Li and C. Liu, "AutoMLP: A Framework for the Acceleration of Multi-Layer Perceptron Models on FPGAs for Real-Time Atrial Fibrillation Disease Detection," in *IEEE Transactions on Biomedical Circuits and Systems*, vol. 17, no. 6
- [2] L. Zhao, C. Liu, S. Wei, Q. Shen, F. Zhou and J. Li, "A new entropy-based atrial fibrillation detection method for scanning wearable ECG recordings", *Entropy*, vol. 20, no. 12, 2018.
- [3] H. Zhang, Z. Dong, J. Gao, P. Lu and Z. Wang, "Automatic screening method for atrial fibrillation based on lossy compression of the electrocardiogram signal", *Physiol. Meas.*, vol. 41, no. 7, 2020
- [4] P. H. Borghi, R. C. Borges and J. P. Teixeira, "Atrial fibrillation classification based on MLP networks by extracting Jitter and Shimmer parameters", *Procedia Comput. Sci.*, vol. 181, pp. 931-939, 2021.
- [5] R. Sarić, D. Jokić, N. Beganović, L. G. Pokvić and A. Badnjević, "FPGA-based real-time epileptic seizure classification using artificial neural network", *Biomed. Signal Process. Control*, vol. 62, 2020.
- [6] <https://www.build-electronic-circuits.com/shift-register/>
- [7] <https://en.wikipedia.org/wiki/File:SinusRhythmLabels.svg>